scaled by the reward returns. In PyTorch, this is defined as `-1 * torch.sum(r * torch.log(preds))`. We compute the loss with the data we've collected for the episode, and run the `torch` optimizer to minimize the loss. Let's run through some actual code.

**Listing 4.6. Computing the discounted rewards**

```
def discount_rewards(rewards, gamma=0.99):
    lenr = len(rewards)
    disc_return = torch.pow(gamma,torch.arange(lenr).float()) *
rewards   1
    disc_return /= disc_return.max()
2
    return disc_return
```

- ***1*** Computes exponentially decaying rewards
- ***2*** Normalizes the rewards to be within the [0,1] interval to improve numerical stability

Here we define a special function to compute the discounted rewards given an array of rewards that would look like `[50,49,48,47,...]` if the episode lasted 50 time steps. It essentially turns this linear sequence of rewards into an exponentially decaying sequence of rewards (e.g., `[50.0000, 48.5100, 47.0448, 45.6041, ...]`), and then it divides by the maximum value to bound the values in the interval `[0,1]`.

The reason for this normalization step is to improve the learning efficiency and stability, since it keeps the return values within the same range no matter how big the raw return is. If the raw return is 50 in the beginning of training but then reaches 200 by the end of training, the gradients are going to change by almost an order of magnitude, which hampers stability. It will still work without normalization, but not as reliably.

#### Backpropagating

Now that we have all the variables in our objective function, we can calculate the loss and backpropagate to adjust the parameters. The following listing shows the loss function, which is just a Python translation of the math we described earlier.